

Experiment 1: 8-Puzzle Problem

Aim

To solve the 8-Puzzle problem using Python.

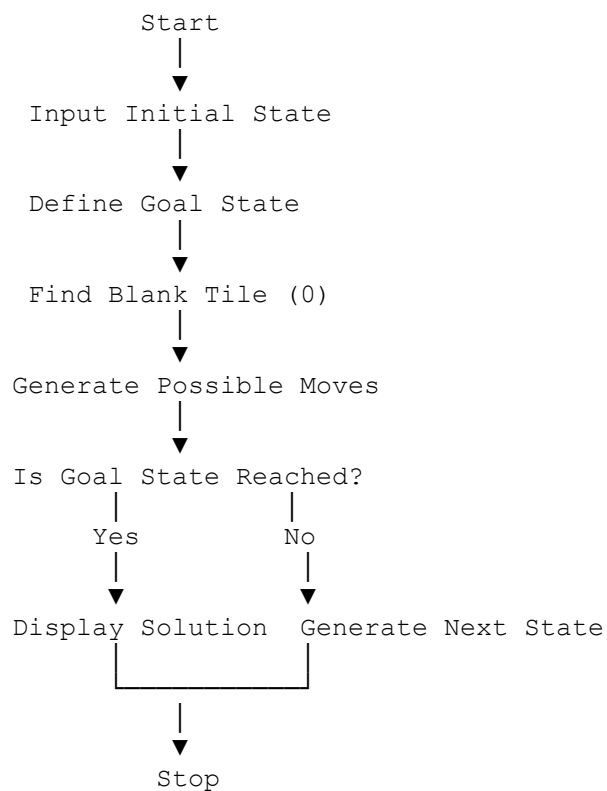
Objective

To arrange the tiles in the correct order by moving the blank space using a search algorithm.

Algorithm

1. Define the initial state of the puzzle.
2. Define the goal state.
3. Find the position of the blank tile (0).
4. Move the blank tile in all possible directions (up, down, left, right).
5. Generate new puzzle states.
6. Repeat until the goal state is reached.
7. Display the solution.

Flowchart



Python Program

```
from collections import deque

goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]

start = [[1,2,3],
         [4,5,6],
         [0,7,8]]

queue = deque([start])
visited = []

while queue:
    state = queue.popleft()

    if state == goal:
        print("Goal State Reached")
        for row in state:
            print(row)
        break

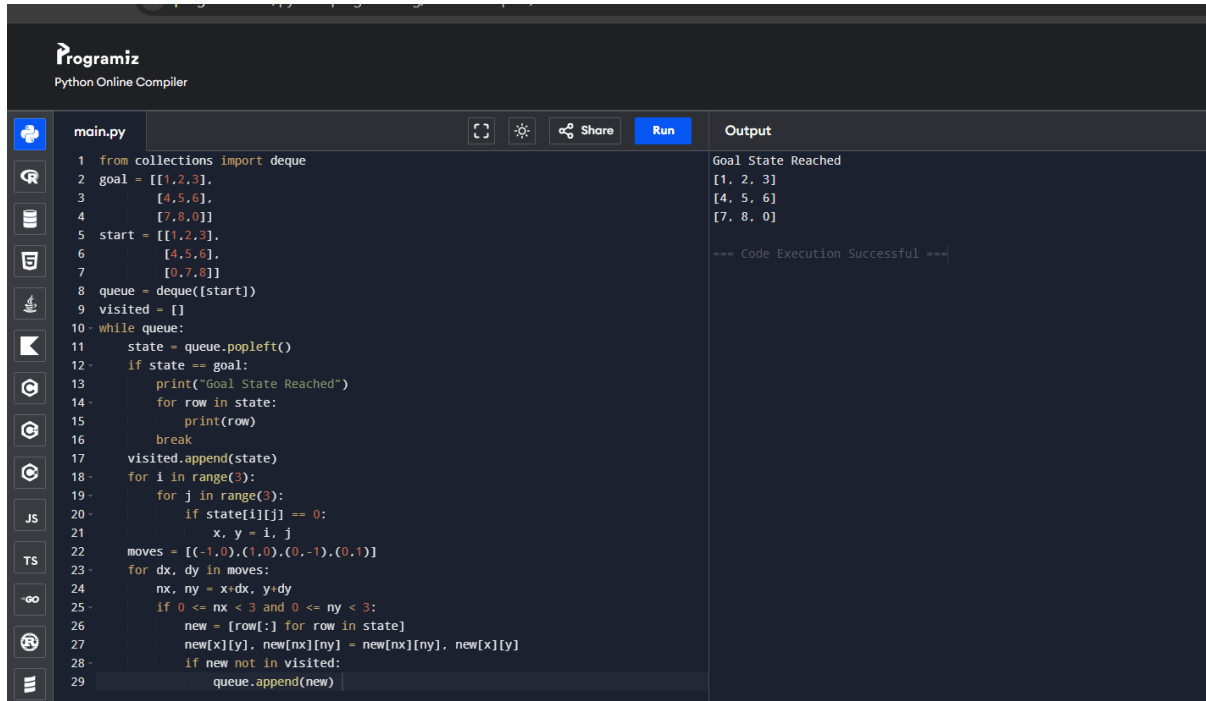
    visited.append(state)

    # Find blank space
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    moves = [(-1,0), (1,0), (0,-1), (0,1)]

    for dx, dy in moves:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new = [row[:] for row in state]
            new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
            if new not in visited:
                queue.append(new)
```

Sample Output



The screenshot displays the Programiz Python Online Compiler interface. The editor shows a Python script named `main.py` that implements a breadth-first search algorithm to solve the 8-Puzzle problem. The script defines a goal state, a start state, and a queue. It iterates through the queue, checking for the goal state and generating new states based on valid moves. The output pane shows the goal state reached and the final state of the puzzle.

```
1 from collections import deque
2 goal = [[1,2,3],
3         [4,5,6],
4         [7,8,0]]
5 start = [[1,2,3],
6         [4,5,6],
7         [0,7,8]]
8 queue = deque([start])
9 visited = []
10 while queue:
11     state = queue.popleft()
12     if state == goal:
13         print("Goal State Reached")
14         for row in state:
15             print(row)
16         break
17     visited.append(state)
18     for i in range(3):
19         for j in range(3):
20             if state[i][j] == 0:
21                 x, y = i, j
22                 moves = [(-1,0),(1,0),(0,-1),(0,1)]
23                 for dx, dy in moves:
24                     nx, ny = x+dx, y+dy
25                     if 0 <= nx < 3 and 0 <= ny < 3:
26                         new = [row[:] for row in state]
27                         new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
28                         if new not in visited:
29                             queue.append(new)
```

Output:

```
Goal State Reached
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

=== Code Execution Successful ===
```

Result

Thus, the 8-Puzzle problem was solved successfully using Python.

Conclusion

The program successfully finds the goal state by moving the blank tile through valid moves. It demonstrates the use of a search algorithm to solve the 8-Puzzle problem.